

FOOTIUM · SECURITY REVIEW

Footium Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Footium	Review	Guardian	November 11 – May 29, 2026	November 24, 2025

§ 00

Contents.

38 pages — overview, confidence ranking, scope, 28 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 21 findings	P. 09
	Remediation Review 1 · 4 findings	P. 30
	Remediation Review 2 · 3 findings	P. 34
§ 05	Disclaimer	P. 37
§ 06	About Guardian	P. 38

§ 01

Overview.

Guardian reviewed Review across 3 review rounds. The final report includes 1 High, 3 Medium, 12 Low, 12 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Footium/footium-eth
CLIENT	METHODOLOGY
Footium	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2
PROJECT	MAIN REVIEW COMMIT
Review	21cd13fec77f
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	638e9ae3640d
AUDIT WINDOW	
November 11 – May 29, 2026	
FINAL REPORT	
November 24, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	1	0	0	0	1
Medium	3	0	2	0	1
Low	12	0	6	0	6
Informational	12	0	7	0	5
All severities	28	0	15	0	13

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `Footium/footium-eth`. The scope includes 113 source lines across 1 file.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/FootiumGeneralMinting.sol	113	203

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Payment Amount Not Multiplied By Mint Quantity	Logical Error	HIGH	RESOLVED	p. 9
M-01	Bypass Of Non-Holder Mints	Logical Error	MEDIUM	ACKNOWLEDGED	p. 10
M-02	Multi-minting DoS'd With Token Id Provided	Logical Error	MEDIUM	RESOLVED	p. 11
M-03	Access Control Not Tied To Ownership	OOS	MEDIUM	ACKNOWLEDGED	p. 12
L-01	Implementation Can Be Initialized Directly	Warning	LOW	RESOLVED	p. 13
L-02	No Validation Against Zero-Amount Mints	Validation	LOW	RESOLVED	p. 14
L-03	Weird Tokens Not Supported	Warning	LOW	RESOLVED	p. 15
L-04	Cross-Chain Replay Attacks	Warning	LOW	ACKNOWLEDGED	p. 16
L-05	Storage Gaps Missing	Warning	LOW	RESOLVED	p. 17
L-06	Per-user Limits Bypassed	Logical Error	LOW	ACKNOWLEDGED	p. 18
L-07	ERC20 Tokens That Do Not Return Boolean Fail	Warning	LOW	RESOLVED	p. 19
L-08	Missing ERC-20 Withdraw Function	Best Practices	LOW	RESOLVED	p. 20
L-09	Merkle Root Update Can Brick User Claims	OOS	LOW	ACKNOWLEDGED	p. 21
L-10	Player ID Uniqueness Check Is Global	OOS	LOW	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-11	Player IDs Are Case-Sensitive	OOS	LOW	ACKNOWLEDGED	p. 23
L-12	Contracts Unable To Receive ETH Price	OOS	LOW	ACKNOWLEDGED	p. 24
I-01	Incorrect NatSpec For parentNftId	Documentation	INFORMATIONAL	RESOLVED	p. 25
I-02	Missing NatSpec For excludeParentNftOwners	Documentation	INFORMATIONAL	RESOLVED	p. 26
I-03	Missing Zero-Address Validation	Warning	INFORMATIONAL	RESOLVED	p. 27
I-04	Parent NFT Ownership Misses Combinations	Documentation	INFORMATIONAL	RESOLVED	p. 28
I-05	Mint Distribution Cannot Be Enforced	Warning	INFORMATIONAL	RESOLVED	p. 29

REMEDIATION REVIEW 1

I-01	Not All Validations For Zero Address Added	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 30
I-02	Missing Event On Native ETH Withdrawal	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 31
I-03	NatSpec Misalignment For mint	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 32
I-04	Multiple Leaves For Same User/Token ID	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 33

REMEDIATION REVIEW 2

I-01	Global mints depend on leaf limits	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 34
I-02	Global count flag should be immutable	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 35
I-03	Payment receiver can be set to zero	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 36

HIGH **H-01**

RESOLVED

Payment Amount Not Multiplied By Mint Quantity

CATEGORY Logical Error
LOCATION FootiumGeneralMinting.sol
REVIEW Main Review

DESCRIPTION

The `mint()` function in `FootiumGeneralMinting` accepts a `value` parameter that represents the payment amount and an `amount` parameter representing the number of NFTs to mint. However, the payment validation logic does not multiply `value` by `amount`, allowing users to mint multiple NFTs while paying only for a single token.

User ends up severely underpaying since users pay a flat value regardless of how many NFTs they mint. The technical documentation in `FootiumGeneralMinting.sol` states:

```
// Setup: Anyone can mint up to 5 NFTs for 0.05 ETH each  
// In merkle tree: [mintId, false, 0, 5, address(0), 0.05 ether]
```

However, the implementation contradicts this by treating `value` as a total payment rather than a per-NFT price.

RECOMMENDATION

Multiply the configured price by amount

RESOLUTION

Status: Resolved.

<https://github.com/Footium/footium-eth/pull/3>

MEDIUM **M-01**

ACKNOWLEDGED

Bypass Of Non-Holder Mints

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/footium-eth-shareable-team1-1762571501016/blob/master/contracts/FootiumGeneralMinting.sol#L150
REVIEW	Main Review

— DESCRIPTION

"Non-holder" mints (`excludeParentNftOwners == true, parentNftId == 0`) only check `_parentNft.balanceOf(receiver)` . A holder can still call mint and forward NFTs to a fresh wallet, bypassing the intent to prevent existing holders from minting.

— RECOMMENDATION

If the goal is to block holders from participating at all, validate `_parentNft.balanceOf(msg.sender) == 0` (or force `receiver == msg.sender`) when exclusion mode is used. Otherwise clarify in docs that the current check only prevents delivery to holder wallets, not holder participation.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Multi-minting DoS'd With Token Id Provided

CATEGORY Logical Error
LOCATION FootiumGeneralMinting.sol: 162
REVIEW Main Review

DESCRIPTION

Multi-minting a fixed tokenId reuses the same identifier inside the mint loop. Unless the underlying IFootiumMintable ignores the parameter (auto-increment mode), any call with amount > 1 and an explicit tokenId will revert on the second iteration, halting the entire mint process.

RECOMMENDATION

Ensure to derive unique IDs per iteration or require that `amount == 1` when `tokenId != 0`.

RESOLUTION

Status: Resolved.

<https://github.com/Footium/footium-eth/pull/5>

MEDIUM **M-03**

ACKNOWLEDGED

Access Control Not Tied To Ownership

CATEGORY OOS
LOCATION Global
REVIEW Main Review

— DESCRIPTION

During initialize the contract grants DEFAULT_ADMIN_ROLE to msg.sender, but transferOwnership does not reassign or revoke that admin role. Once the owner hands the contract to a multisig or new team, the previous owner still has full admin authority (can grant or revoke MINTER_ROLE, change other admins, etc.), while the new owner cannot manage minters because DEFAULT_ADMIN_ROLE never moved.

— RECOMMENDATION

Update ownership transfers (or add a setter) so AccessControl admins mirror the current owner, and revoke the old admin when ownership changes.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Implementation Can Be Initialized Directly

CATEGORY Warning
LOCATION Global
REVIEW Main Review

DESCRIPTION

None of the upgradeable contracts disable their initializers on the implementation contract. An attacker can call initialize on the implementation directly and become the implementation owner/admin. While this does not affect proxies directly, it enables phishing and operator confusion, and can be dangerous if the implementation ever receives ETH or is otherwise interacted with by mistake.

RECOMMENDATION

Add a constructor that calls `_disableInitializers()` in each upgradeable contract to lock the implementation.

RESOLUTION

Status: Resolved.

<https://github.com/Footium/footium-eth/pull/6>

LOW **L-02**

RESOLVED

No Validation Against Zero-Amount Mints

CATEGORY	Validation
LOCATION	FootiumGeneralMinting.sol
REVIEW	Main Review

— DESCRIPTION

The mint() function in FootiumGeneralMinting does not validate that amount > 0 before processing payment. A user can successfully call the function with amount = 0 while providing a non-zero value, resulting in payment being transferred to the payment receiver without any NFTs being minted to the user. The payment logic executes regardless of the amount, while the minting loop only executes if amount > 0. This creates a scenario where users can lose funds without receiving any tokens in return.

— RECOMMENDATION

Add validation to require non-zero amount.

— RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Weird Tokens Not Supported

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The FootiumPrizeDistributor and FootiumGeneralMinting contracts assume that ERC20 token transfers move the exact amount specified. This assumption breaks for tokens with non-standard transfer mechanics.

— RECOMMENDATION

Document that only standard ERC20 tokens are supported

— RESOLUTION

Status: Resolved.

<https://github.com/Footium/footium-eth/pull/8>

LOW **L-04**

ACKNOWLEDGED

Cross-Chain Replay Attacks

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The merkle leaf construction in FootiumGeneralMinting, FootiumAcademy, and FootiumPrizeDistributor contracts does not include any chain-specific identifier such as block.chainid or a deployment salt. The leaf hashes only contain mint parameters like addresses, amounts, and IDs. If the protocol deploys the same merkle root across multiple chains for operational convenience, users can replay their valid merkle proofs on every chain where the contract is deployed.

— RECOMMENDATION

Include a chain id for the proof if the plan is to go cross-chain.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Storage Gaps Missing

CATEGORY	Warning
LOCATION	FootiumGeneralMinting.sol
REVIEW	Main Review

— DESCRIPTION

FootiumGeneralMinting is an upgradeable contract (PausableUpgradeable, ReentrancyGuardUpgradeable, OwnableUpgradeable) but does not reserve storage slots for future variables. Without a `_gap` array, any future version that adds state variables will shift inherited storage layout and can corrupt parent storage.

— RECOMMENDATION

Consider appending a gap to storage.

— RESOLUTION

Status: Resolved.

LOW **L-06**

ACKNOWLEDGED

Per-user Limits Bypassed

CATEGORY	Logical Error
LOCATION	FootiumGeneralMinting.sol: 258
REVIEW	Main Review

— DESCRIPTION

Per-user limits are keyed to receiver, not to the caller: `_checkMintLimit` updates `_mintCounts[mintId]` [receiver] and the Merkle leaf omits receiver whenever `isAddressReserved` is false. A single account can therefore reuse the same proof indefinitely simply by changing the receiver argument to fresh addresses they control, defeating “per user” or “per proof” caps while still paying once per call. An arbitrary user can also intentionally mint to a victim’s address, consuming that victim’s entire mint allowance (although there may be little incentive to do this as the user would pay for the victim’s mint).

— RECOMMENDATION

Consider if this is expected behavior for the protocol. If not, either bind the receiver into the proof even for public mints or track limits by `msg.sender`.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

ERC20 Tokens That Do Not Return Boolean Fail

CATEGORY Warning
LOCATION FootiumGeneralMinting.sol: 282
REVIEW Main Review

DESCRIPTION

Function `_handlePayment()` assumes all ERC20 tokens return a boolean value from `transferFrom()`. However, some widely-used tokens like USDT (on Ethereum mainnet) do not return any value from their `transfer()` and `transferFrom()` functions. When Solidity attempts to decode an empty return value as a bool, the transaction will revert with an ABI decoding error.

RECOMMENDATION

Consider using `SafeERC20` or set the `_paymentToken` carefully.

RESOLUTION

Status: Resolved.

LOW **L-08**

RESOLVED

Missing ERC-20 Withdraw Function

CATEGORY	Best Practices
LOCATION	FootiumGeneralMinting.sol
REVIEW	Main Review

DESCRIPTION

The contract currently supports receiving both native ETH and ERC-20 tokens. However, only native ETH can be recovered via the existing `withdraw` function. If ERC-20 tokens are accidentally transferred to the contract (for example, by users sending tokens directly), those tokens remain permanently stuck since there is no recovery mechanism for them.

RECOMMENDATION

Add an emergency withdrawal function for arbitrary ERC-20 tokens, restricted to owner role.

RESOLUTION

Status: Resolved.

LOW **L-09**

ACKNOWLEDGED

Merkle Root Update Can Brick User Claims

CATEGORY OOS
LOCATION FootiumPrizeDistributor
REVIEW Main Review

— DESCRIPTION

The FootiumPrizeDistributor contract allows the owner to update the merkle root at any time via `setERC20MerkleRoot()` and `setETHMerkleRoot()`. The claim functions calculate the claimable amount as:

```
uint256 value = _amount - totalERC20Claimed[_token][_to];
```

If the owner updates the merkle root with a cumulative `_amount` that is lower than what the user has already claimed `(totalERC20Claimed[_token][_to])`, the subtraction will underflow and revert, blocking user claims.

— RECOMMENDATION

Add validation to ensure merkle root updates don't reduce cumulative amounts below already-claimed totals.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Player ID Uniqueness Check Is Global

CATEGORY OOS
LOCATION FootiumAcademy.sol
REVIEW Main Review

DESCRIPTION

The FootiumAcademy contract tracks minted players using a mapping keyed only by playerId:

```
mapping(string => bool) private _mintedPlayers;
```

The contract checks `require(_mintedPlayers[playerId] == false, "Player already minted");` so once the id is minted, it cannot be minted again.

However, the merkle proof verification includes both clubId and playerId: `bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(clubId, playerId, msg.value))));`

The off-chain merkle tree can legitimately contain multiple entries with the same playerId but different clubId values. However, once any club mints a player with a given playerId, no other club can mint a player with that same ID, even if they have a valid merkle proof.

RECOMMENDATION

Change the _mintedPlayers mapping to be keyed by both clubId and playerId.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Player IDs Are Case-Sensitive

CATEGORY OOS
LOCATION FootiumAcademy.sol
REVIEW Main Review

— DESCRIPTION

The FootiumAcademy contract uses string type for player IDs without any normalization or case-insensitive comparison. The `_mintedPlayers` mapping stores player IDs exactly as provided, meaning "Player-123", "player-123", and "PLAYER-123" are treated as three completely different players. The merkle proof verification encodes the player ID string as-is, so case variations produce different merkle leaves and are considered valid distinct entries. If the off-chain merkle tree generation system doesn't enforce consistent casing, the same logical player can be minted multiple times with different case variations.

— RECOMMENDATION

Ensure the off-chain system normalizes player ID's and does not multiple entries for the same player.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

Contracts Unable To Receive ETH Prize

CATEGORY OOS

LOCATION FootiumPrizeDistributor.sol: 185

REVIEW Main Review

— DESCRIPTION

The claimETHPrize function transfers ETH using a low-level call without providing an alternative claim mechanism. The function enforces that only the prize recipient can claim (`_to != msg.sender` check), and if the recipient is a contract address without a `receive()` or `payable fallback()` function, the ETH transfer will always fail, locking the funds for the recipient.

— RECOMMENDATION

Clearly document this behavior.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

Incorrect NatSpec For parentNftId

CATEGORY	Documentation
LOCATION	FootiumGeneralMinting.soll
REVIEW	Main Review

DESCRIPTION

The NatSpec for `mint` states `parentNftId ... (0 or type(uint256).max to skip ownership check)`. However, the implementation only treats `parentNftId == 0` as the skip condition. If the backend follows the documented `type(uint256).max` sentinel, calls will reach the `ownerOf(parentNftId)` branch and revert, permanently disabling those mint configurations.

RECOMMENDATION

Update the NatSpec or adjust the ownership check code in `mint` to be synchronized.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Missing NatSpec For excludeParentNftOwners

CATEGORY Documentation
LOCATION FootiumGeneralMinting.sol
REVIEW Main Review

DESCRIPTION

In the `mint` function, the `excludeParentNftOwners` input variable does not have a NatSpec, unlike the other variables.

RECOMMENDATION

Add a NatSpec `@param excludeParentNftOwners` entry that clearly explains the two behaviors (require ownership vs. exclude owners).

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-03**

RESOLVED

Missing Zero-Address Validation

CATEGORY	Warning
LOCATION	FootiumGeneralMinting.sol: 52-68
REVIEW	Main Review

DESCRIPTION

The initialize() function in FootiumGeneralMinting does not validate that the provided addresses are non-zero before assigning them to critical contract state variables.

RECOMMENDATION

Add zero-address validation for critical parameters in the initialize() function

RESOLUTION

Status: Resolved.

INFORMATIONAL I-04

RESOLVED

Parent NFT Ownership Misses Combinations

CATEGORY	Documentation
LOCATION	FootiumGeneralMinting.sol
REVIEW	Main Review

DESCRIPTION

The `_checkParentNftOwnership()` function only validates two specific combinations of `parentNftId` and `excludeParentNftOwners` parameters, leaving other combinations unchecked:

- (1) `parentNftId != 0 AND excludeParentNftOwners == true`
- (2) `parentNftId == 0 AND excludeParentNftOwners == false`

The intent of these parameter combinations are ambiguous. For example, if `excludeParentNftOwners` is true, the contract can enforce that the parent NFT balance of the receiver is 0, but that would be contradictory with the defined `parentNftId` having to be owned by a user.

RECOMMENDATION

Clearly document these no-op combinations.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

Mint Distribution Cannot Be Enforced

CATEGORY	Warning
LOCATION	FootiumGeneralMinting.sol
REVIEW	Main Review

DESCRIPTION

The FootiumGeneralMinting contract cannot enforce transaction-level pacing or batch-size restrictions as the amount parameter is not included in the merkle proof. A user can mint their entire allocation in a single transaction (amount = limit) or split it across multiple calls.

RECOMMENDATION

Consider if this is okay for the purposes for the general minting contract and clearly document this behavior.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Not All Validations For Zero Address Added

CATEGORY Best Practices
LOCATION FootiumGeneralMinting.sol: 61
REVIEW Remediation Review 1

DESCRIPTION

Parameters `parentNft` and `paymentToken` are not checked against the zero address unlike `mintableNft` and `paymentReceiverAddress`.

RECOMMENDATION

Consider adding zero address checks for those variables as well.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Missing Event On Native ETH Withdrawal

CATEGORY Best Practices
LOCATION FootiumGeneralMinting.sol
REVIEW Remediation Review 1

DESCRIPTION

The recoverERC20 function emits an ERC20Recovered event, but the native ETH withdraw path does not emit any event.

RECOMMENDATION

Consider emitting a dedicated event for native ETH withdrawals to ensure parity with ERC20 recovery and improving observability for all recovery actions.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

NatSpec Misalignment For mint

CATEGORY	Best Practices
LOCATION	FootiumGeneralMinting.sol
REVIEW	Remediation Review 1

DESCRIPTION

The NatSpec on `mint: excludeParentNftOwners` currently describes only one way to bypass the parent-NFT ownership check, but `_checkParentNftOwnership` actually supports multiple skip conditions. This discrepancy can mislead integrators into assuming ownership is enforced in cases where it is not.

Likewise, the `value` parameter is documented only as "The payment amount," even though the total cost is computed as `value * amount`, which may cause confusion.

RECOMMENDATION

Simplify the NatSpec so it does not state the specific bypass combinations. Instead, keep the high-level description in `mint` general, and rely on the detailed four-case commentary already present inside `_checkParentNftOwnership` to define the exact behavior.

Also clarify in the NatSpec that `value` represents the per-NFT price, not the total payment.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Multiple Leaves For Same User/Token ID

CATEGORY	Warning
LOCATION	FootiumGeneralMinting.sol
REVIEW	Remediation Review 1

DESCRIPTION

It is possible for the same receiver to have multiple leaves and for the same tokenId but with different values. Consequently, a user can claim a specific NFT but for a cheaper price than encoded in another leaf. Furthermore, with multiple leaves and a specific tokenId, only the first user to mint will be able to receive that specific token.

RECOMMENDATION

Clearly document this behavior and ensure the Merkle tree is appropriately constructed to spec.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Global mints depend on leaf limits

CATEGORY Warning
LOCATION FootiumGeneralMinting.sol
REVIEW Remediation Review 2

DESCRIPTION

In global mint-count mode, uniqueness is only enforced if the Merkle leaf uses limit = 1. If limit = 0, the mint is unlimited and untracked; if limit > 1, the same global `mintId` can be minted multiple times.

RECOMMENDATION

Enforce limit = 1 for global mints (backend), and add contract-level protection to reject limit = 0 or limit > 1 when `useGlobalMintCounts` is enabled.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Global count flag should be immutable

CATEGORY Best Practices
LOCATION FootiumGeneralMinting.sol
REVIEW Remediation Review 2

DESCRIPTION

`useGlobalMintCounts` is intended to be a deployment-time mode flag, but it is currently stored as a normal storage variable.

RECOMMENDATION

Make `useGlobalMintCounts` `immutable`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Payment receiver can be set to zero

CATEGORY	Validation
LOCATION	FootiumGeneralMinting.sol
REVIEW	Remediation Review 2

DESCRIPTION

`setPaymentReceiverAddress` does not reject `address(0)`, even though the constructor rejects a zero payment receiver. If the owner accidentally sets the payment receiver to zero, future mints will forward payments to `address(0)`.

RECOMMENDATION

Add the same zero-address validation to `setPaymentReceiverAddress`.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.